

Parcours de Validation du Moteur ExoSync

But : valider le **noyau** d'ExoSync — moteur + parsing/exécution MXL sur de vrais fichiers Excel — *avant* de toucher à la création d'écosystèmes ou aux apps web.

Principe : un escalier. Chaque marche = une manip Excel que tu fais à la main, une commande, et un **critère** « **je l'ai vu marcher de mes yeux** ». On ne monte à la marche suivante que quand la précédente est validée (OK dans le classeur de suivi).

Public : ce document est écrit pour qu'une **personne extérieure au projet** puisse le suivre seule. Si elle bloque quelque part, c'est que le noyau n'est pas assez clair.

Compagnon de ce document : Validation-Moteur-ExoSync.xlsx (classeur de suivi avec résultats attendus / obtenus / statut). Le code testé est dans C:\Users\fabie\Documents\JLC\Python\SysEng.

Vue d'ensemble de l'escalier

#	Marche	Ce qu'on valide	Concepts MXL introduits
0	Reprendre la main sur l'outillage	Lancer le moteur + la démo, lire un Dashboard	(aucun)
1	Un fichier se décrit et pousse	Excel -> une clé dans le store	FILE_TYPE, FILE_ID, DEF ... GET_TABLE, PUSH
2	Deux fichiers communiquent	Les lignes de A apparaissent dans B	PULL ... FILL_TABLE MODE=OVERWRITE
3	B calcule et affiche	B montre 70 % dans son Dashboard	COMPUTE(AVG), BIND
4	Accumuler plusieurs sources	B combine A1 + A2 sans doublon	MODE=APPEND_NEW KEY=
5	Trois niveaux (la démo entière)	UO -> Cockpit -> Pilote compris	tout le noyau assemblé
6+	La suite (hors noyau)	pointeurs seulement	VALIDATE, feux, LIST/COLLECT, web

Rappels valables pour TOUTES les marches

Comment créer une table Excel nommée

1. Tape les en-têtes en ligne 1, les données en dessous.
2. Sélectionne tout le bloc (en-têtes + données).
3. Ruban **Insertion** -> **Tableau** (raccourci Ctrl + L). Coche « **Mon tableau comporte des en-têtes** ».
4. Le tableau étant sélectionné : ruban **Création de tableau** -> **Nom du tableau** (à gauche), et tape le nom exact demandé (ex. tbl_act).

Comment créer une plage nommée (cible d'un `BIND`)

1. Clique sur la cellule (ex. Dashboard!F3).
2. Dans la **Zone Nom** (case à gauche de la barre de formule), tape le nom demandé (ex. avancement) puis **Entrée**.

Comment créer la feuille `_Manifeste`

1. Crée une feuille nommée **exactement** `_Manifeste` (avec l'underscore, M majuscule).
2. Cellule **A1** = `MANIFESTE_V=1`
3. Ligne **2** : libre (mets instruction en A2 si tu veux, le moteur l'ignore).
4. À partir de **A3**, une instruction MXL **par cellule**, en **colonne A**, dans l'ordre. Les lignes vides et les lignes commençant par `#` (commentaires) sont autorisées.

Commandes (terminal ouvert dans le dossier `sysEng`)

```
# Une seule fois : installer les dépendances
pip install -r requirements.txt

# Vider le store (à faire avant de recommencer une marche proprement)
python -m src store clear

# Valider UN fichier fait main (le cœur de ce parcours)
python scripts/valider_un.py validation/NOM_DU_FICHER.xlsx

# Voir le contenu du store
python -m src status
```

[!] **Ferme toujours le fichier Excel dans Excel avant de lancer une commande** qui écrit dedans (sinon « fichier verrouillé »). Rouvre-le ensuite pour constater le résultat.

[dossier] Crée un dossier `validation/` à la racine de `sysEng` pour y ranger tes fichiers de test.

Marche 0 — Reprendre la main sur l'outillage

Objectif : prouver que tu sais lancer le moteur, exécuter la démo complète, et lire un résultat dans un Dashboard Excel. Aucun fichier à créer.

Étapes 1. Ouvre un terminal dans `C:\Users\fabie\Documents\JLC\Python\SysEng`. 2. `pip install -r requirements.txt` (une seule fois). 3. `python scripts/generate_demo.py ->` génère 7 Excel dans `output/demo/`. 4. `python scripts/sync_demo.py ->` synchronise tout. 5. Ouvre `output/demo/COCKPIT-ALICE.xlsx`, onglet **Dashboard**.

Résultat attendu - La console affiche `[OK] Synchronisation demo terminée SANS erreur.` - Dans `COCKPIT-ALICE.xlsx`, onglet **Dashboard** : `Avancement global (cellule F5) = 55 %`.

Critère de validation : console sans erreur **ET** `F5 = 55 %`.

Tu as le droit d'ignorer : tout le reste (apps web, LIST/COLLECT, historique...).

Marche 1 — Un fichier se décrit et pousse une donnée

Objectif : un fichier Excel se présente au moteur et **envoie** sa table d'activités dans le store central. La plus petite unité de « communication » : fichier -> store.

Fichier à créer : `validation/m1_source.xlsx`

Feuille `Activites` — crée une table nommée `tbl_act` :

id	libelle	avancement
A1	Analyse fonctionnelle	80
A2	Rédaction spécification	60

Feuille `_Manifeste` (A1 = `MANIFESTE_V=1`, puis à partir de A3) :

```
FILE_TYPE: uo_instance
FILE_ID: TEST-SOURCE

# Je lis ma propre table d'activités
DEF $act = GET_TABLE(Activites, tbl_act)

# Je publie cette table dans le store central
PUSH $act -> test.source.activites
```

Test

```
python -m src store clear
python scripts/valider_un.py validation/m1_source.xlsx
```

Résultat attendu (console)

```
FILE_TYPE : uo_instance
FILE_ID   : TEST-SOURCE
PUSH ...  : 1
    -> test.source.activites
Store : test.source.activites -> [2 lignes]
[OK] Execution terminée sans erreur.
```

Critère de validation : `PUSH = 1`, clé `test.source.activites` présente avec **2 lignes**, zéro erreur.

Ce que tu viens de prouver : le moteur lit une feuille `_Manifeste`, comprend `FILE_TYPE/DEF/PUSH`, lit une vraie table Excel et la dépose dans le store.

Marche 2 — Deux fichiers communiquent

Objectif : un **deuxième** fichier (B) va **recupérer** la table déposée par A et l'écrire dans une de ses feuilles. Tu ouvriras B et tu **verras les lignes de A** apparaître. C'est le cœur de ta demande : *la communication entre fichiers Excel, ressentie*.

On réutilise `m1_source.xlsx` (le fichier A) tel quel.

Fichier à créer : `validation/m2_recepteur.xlsx` (le fichier B)

Feuille Donnees — crée une table nommée **tbl_recue** avec les **mêmes en-têtes** et **une ligne d'attente** (la table doit exister pour être remplie) :

id	libelle	avancement
-	(en attente de synchro)	0

Feuille _Manifeste :

```
FILE_TYPE: cockpit
FILE_ID: TEST-RECEPTEUR

# Je récupère la table publiée par le fichier A et je remplis ma feuille Donnees
PULL test.source.activites -> FILL_TABLE(Donnees, tbl_recue) MODE=OVERWRITE
```

Test (l'ordre compte : A pousse AVANT que B tire)

```
python -m src store clear
python scripts/valider_un.py validation/m1_source.xlsx      # A pousse
python scripts/valider_un.py validation/m2_recepteur.xlsx   # B tire
```

Puis ouvre **m2_recepteur.xlsx**, onglet **Donnees**.

Résultat attendu - Console du 2■ appel : **PULL ... : 1**, zéro erreur. - Dans Excel, la feuille Donnees contient maintenant **les 2 lignes de A** (A1 / Analyse fonctionnelle / 80 — A2 / Rédaction spécification / 60). La ligne « (en attente de synchro) » a disparu (mode **OVERWRITE**).

Critère de validation : les données de A sont **physiquement visibles** dans le fichier B.

[?] **Dépannage** — « **PULL = 0** » sans erreur ? Cela veut dire que la clé demandée n'existe pas (encore) dans le store. Le moteur ne plante pas volontairement (dégradation gracieuse : un fichier absent ne casse pas les autres). Causes probables : tu as lancé B **avant** A, tu as vidé le store entre les deux, ou la clé du **PULL** ne correspond pas exactement à celle du **PUSH**. Vérifie avec `python -m src status`.

Ce que tu viens de prouver : deux fichiers Excel indépendants échangent des données via le store, sans connexion directe entre eux. **C'est l'exostructure en action**.

Marche 3 — Le récepteur calcule et affiche

Objectif : B ne fait plus que recopier — il **calcule** une moyenne sur les données reçues et **l'affiche** dans son Dashboard.

Modifier **validation/m2_recepteur.xlsx**

Ajoute une feuille Dashboard : - En E3 : tape **Avancement moyen**. - En F3 : laisse vide (le moteur écrira ici). Crée la **plage nommée avancement** sur Dashboard!F3.

Complète la feuille _Manifeste (ajoute ces lignes à la suite) :

```
# Je relis la table que je viens de recevoir
DEF $recu = GET_TABLE(Donnees, tbl_recue)

# Je calcule la moyenne de la colonne avancement
```

```
DEF $moy = COMPUTE(AVG($recu.avancement))

# J'affiche le résultat dans mon Dashboard
BIND $moy -> Dashboard.avancement
```

Test

```
python -m src store clear
python scripts/valider_un.py validation/m1_source.xlsx
python scripts/valider_un.py validation/m2_recepteur.xlsx
```

Puis ouvre `m2_recepteur.xlsx`, onglet **Dashboard**, cellule **F3**.

Résultat attendu - Console : `BIND ... : 1`, zéro erreur. - `Dashboard!F3 = 70` (moyenne de 80 et 60).

Critère de validation : F3 affiche **70**.

Ce que tu viens de prouver : `GET_TABLE`, `COMPUTE(AVG)` et `BIND` fonctionnent — le moteur calcule en Python et réécrit le résultat dans Excel.

Marche 4 — Accumuler plusieurs sources sans doublon

Objectif : B agrège **deux** sources (A1 + A2). Le second **PULL** **ajoute** les lignes au lieu de tout écraser, et **KEY** empêche les doublons.

Fichier à créer : `validation/m4_source2.xlsx` (une **2** source)

Feuille `Activites` — table nommée `tbl_act2` :

id	libelle	avancement
A3	Plan de test	50
A4	Exécution recette	30

Feuille `_Manifeste` :

```
FILE_TYPE: uo_instance
FILE_ID: TEST-SOURCE-2

DEF $act = GET_TABLE(Activites, tbl_act2)
PUSH $act -> test.source2.activites
```

Modifier `validation/m2_recepteur.xlsx`

Ajoute un **second PULL** dans `_Manifeste`, **juste après le premier** :

```
# Première source : on écrase
PULL test.source.activites -> FILL_TABLE(Donnees, tbl_recue) MODE=OVERWRITE
# Deuxième source : on ajoute les nouvelles lignes (clé = id pour éviter les doublons)
PULL test.source2.activites -> FILL_TABLE(Donnees, tbl_recue) MODE=APPEND_NEW KEY=id
```

(garde les lignes `DEF/COMPUTE/BIND` de la marche 3 en dessous)

Test

```
python -m src store clear
python scripts/valider_un.py validation/m1_source.xlsx
python scripts/valider_un.py validation/m4_source2.xlsx
python scripts/valider_un.py validation/m2_recepteur.xlsx
```

Puis ouvre `m2_recepteur.xlsx`.

Résultat attendu - Feuille Données : **4 lignes** (A1, A2, A3, A4). - Dashboard!F3 = **55** (moyenne de 80, 60, 50, 30). - Si tu relances `m2_recepteur.xlsx` une 2^{ème} fois sans vider le store : **toujours 4 lignes** (pas de doublon, grâce à `KEY=id`).

Critère de validation : 4 lignes, F3 = 55, et pas de doublon en relançant.

Ce que tu viens de prouver : `APPEND_NEW + KEY` — l'accumulation idempotente, brique de toute consolidation.

Marche 5 — Trois niveaux : comprendre la démo entière

Objectif : tu maîtrises maintenant chaque brique. La démo officielle n'est que ça, en plus grand : 4 UO -> 2 Cockpits -> 1 Pilote.

Étapes 1. `python -m src store clear` 2. `python scripts/generate_demo.py` 3. `python scripts/sync_demo.py`
4. Ouvre les `_Manifeste` de `output/demo/COCKPIT-ALICE.xlsx` et `output/demo/PILOTE.xlsx`.

Résultat attendu - COCKPIT-ALICE : UO-A1 = 70 %, UO-A2 = 40 %, Global = 55 %. - COCKPIT-BRUNO : UO-B1 = 80 %, UO-B2 = 15 %, Global = 47,5 %. - PILOTE : Alice = 55 %, Bruno = 47,5 %.

Critère de validation : tu peux **expliquer chaque ligne** des Manifestes Cockpit/Pilote en pointant la marche (1 à 4) qui l'a introduite. Si oui -> **le noyau est validé**. [OK]

Marche 6+ — La suite (hors noyau, à ne PAS attaquer avant que 0->5 soient verts)

Une fois le noyau verrouillé, on enrichira *une fonctionnalité à la fois*, toujours selon le même protocole (manip -> commande -> résultat vu) :

- **VALIDATE** — règles de qualité (`RANGE`, `NOT_NULL`, `IN...`). Voir [\[\[Specs/MXL-guide-non-tech\]\]](#).
- **Indicateurs** — `TRAFFIC_LIGHT`, `SWITCH_RANGE` (feux rouge/orange/vert).
- **GROUP_BY / filtres** — agrégation par groupe.
- **LIST + COLLECT** — écosystème hiérarchique. Voir [\[\[Specs/MXL-ecosystem-list-collect\]\]](#).
- **Apps web N1/N2** — *seulement* quand le moteur en ligne de commande est une évidence pour toi.

-> Quand on y arrivera, on bascule sur le chantier « **validation de la création d'écosystèmes** ».

Liens

- [\[\[_INDEX\]\]](#) — index du projet ExoSync
- [\[\[Specs/MXL-guide-non-tech\]\]](#) — référence « Je veux faire X »

- [[06-Roadmap]] — roadmap globale (les 13 modules)
- Code : C:\Users\fabie\Documents\JLC\Python\SysEng
- Lanceur de test : scripts/valider_un.py
- Classeur de suivi : Validation-Moteur-ExoSync.xlsx